

139. Compter ce que coûte une boucle

Le compilateur ne cache pas ses prix.

1. Ce qu'il faut comprendre

Compiler donne son compte rendu : octets de programme, fonctions, variables, mémoire de travail. **Changer une ligne et recompiler montre ce qu'elle a coûté** — c'est la seule mesure qui ne mente pas.

Deux choses à savoir, mesurées et non supposées. `100 * x` et `x * 100` ne coûtent pas la même chose : la routine générale boucle sur son opérande de droite, alors le compilateur met le nombre connu à droite quand l'opérateur s'en moque.

Et `x / 16` devient quatre décalages, `x % 16` un `and` — **quand le diviseur est connu**. Sinon, c'est la routine générale, et elle se paie.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **545 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
total()         42 o ← nombre() - ta fonction
main()          29 o ← démarrage - ta fonction
VBlank          8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← démarrage - attend que l'écran soit libre
AttendreAcces   12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EcrireTexte     10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre    33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles    67 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
Tuiles          224 o ← CopierTuiles - 14 lettres sur 43
"TOTAL"         5 o ← texte() - un texte gravé
TABLE_0         8 o ← total() - une table gravée
```

Lettres gravées : AL0T0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 1    11 o sur place, dans main() + EcrireTexte (10 o)
nombre() × 1    8 o sur place, dans main() + total() (42 o), EcrireNombre (33 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
total() × 1     3 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : additionne les 8 nombres d'une table et affiche le total.
//
// Le but de la leçon : compiler, puis lire le compte rendu (octets de programme,
// mémoire...). Changer une ligne et recompiler montre ce qu'elle coûte.

#include <texte>    // écrit un texte à l'écran
#include <nombre>  // écrit un nombre en chiffres
```

2. UNE TABLE GRAVÉE DANS LA CARTOUCHE — TABLE

```
const uint8_t TABLE[] = { 3, 1, 4, 1, 5, 9, 2, 6 }; // 8 nombres gravés dans la cartouche.
```

3. UNE FONCTION — TOTAL()

```
uint8_t total() {                                // Rend la somme de la table.
    uint8_t somme = 0;                            // On part de 0...
    for (uint8_t i = 0; i < sizeof(TABLE); i++) somme += TABLE[i];
                                                // ... et on ajoute chaque case (sizeof = 8 cases).
                                                // 3 + 1 + 4 + 1 + 5 + 9 + 2 + 6 = 31.

    return somme;
}
```

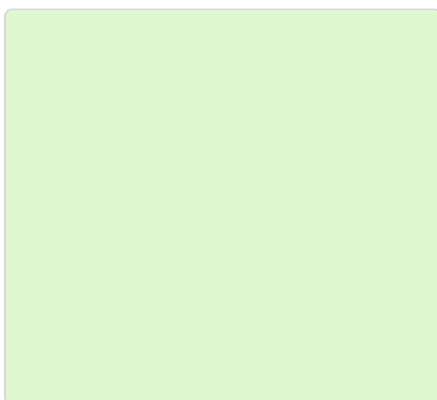
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                    // Le jeu commence ici.
    texte(2, 6, "TOTAL");
    nombre(10, 6, total());                    // Affiche 031.

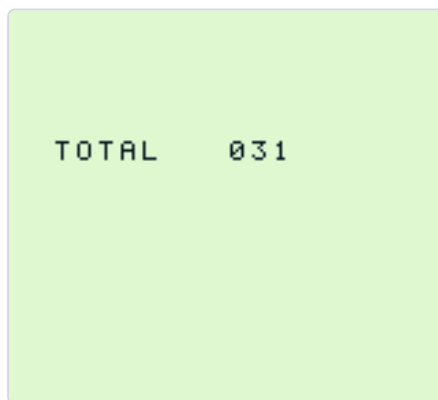
    while (true) {                             // Répète pour toujours :
        image();                               // attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la somme est juste

✓ l'étiquette est là

Ensuite : **140. Le même jeu, deux fois** — chapitre 24 : Aller au bout. · [gameboy3](#) — écrire une cartouche Game Boy en C++